

Reconstructing Benchmark Evaluations: A Public-Artifact Audit of Foundation-Model Benchmarks for Data Science

Le Thi Hong Cuc

Ho Chi Minh City University of Technology (HCMUT), Vietnam National University Ho Chi Minh City
Ho Chi Minh City, Vietnam

Abstract

Benchmarks function as measurement instruments for foundation models, yet a leaderboard score is only as trustworthy as the public evidence that lets an independent party re-run the evaluation. We ask a narrow, checkable question: given only a benchmark’s paper and public artifacts, can a third party reconstruct the evaluation protocol and execute the official evaluator without foundation-model inference? We screen 35 benchmark candidates and audit a **primary frozen cohort of 26 releases**—spanning code generation, text-to-SQL and database agents, tabular and spreadsheet analysis, machine-learning engineering, and end-to-end data-science agents—each pinned to a repository commit and a single audit date (13 July 2026). We separate three outcomes that prior benchmark-quality work tends to conflate: *accessibility* (is the described artifact publicly reachable), *reconstructability* (can the consequential evaluation details be instantiated without unsupported guesses), and *evaluator executability* (does the official scorer run on CPU over supplied or schema-valid synthetic predictions). Three heterogeneous, independently operated model judges label six evidence-grounded criteria per release; deterministic aggregation, agreement, leave-one-judge-out, and repeatability analyses are reported. Every release is then assessed under a pre-specified CPU-only, two-pass protocol (unassisted, then minimal repair) and a fixed resource cap. Official repository resolution succeeds for **30/35 candidates (85.7%)**; four repositories found after outcomes were known form a declared expansion sensitivity. Within the primary cohort, three-model majority coding classifies **13/26 (50.0%)** as reconstructable under a permissive rule, but nominal inter-model agreement misses its pre-specified target. For the less discretionary execution endpoint, **5/26 (19.2%, 95% CI [8.5%, 37.9%])** run as documented; two more run only after documented repairs. The expansion yields similar descriptive results (R2 16/30; first-pass E3 8/30; repaired E3 10/30). The dominant reconstruction gap is not missing code but undocumented predictions-to-score mapping and grading rules; the dominant execution gap is runtime dependence on credentials, network services, or absent required data. We release the corpus registry, line-addressable evidence packets, prompts, raw and aggregated model judgments, execution logs, minimal-repair patches, and the analysis pipeline as a reusable resource for benchmark authors, users, and reviewers (<https://github.com/nausicaa2701/reconstructing-benchmark-evaluations>). These conditional estimates measure public evaluation reconstructability; they are neither population prevalence estimates nor a ranking of benchmark quality.

CCS Concepts

• **Computing methodologies** → **Machine learning**; • **Information systems** → **Data mining**; • **Software and its engineering** → **Software reliability**.

Keywords

benchmark auditing, evaluation reproducibility, foundation models, data science, artifact evaluation, meta-evaluation, reproducibility

ACM Reference Format:

Le Thi Hong Cuc. 2027. Reconstructing Benchmark Evaluations: A Public-Artifact Audit of Foundation-Model Benchmarks for Data Science. In *Proceedings of Proceedings of the 33rd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2027)*. ACM, New York, NY, USA, 10 pages.

1 Introduction

Benchmarks are the measurement instruments of foundation-model research: a reported score on DS-1000 [11] or MLE-bench [1] is routinely read as evidence that a model can write data-science code or carry out machine-learning engineering. As with any instrument, the value of a benchmark score depends on whether an independent party can inspect how it was produced and obtain the same reading from the same procedure. For foundation models this is unusually hard: the headline number couples a model, an agent scaffold, a dataset, and a scoring pipeline, and reproducing it end to end can require substantial inference cost and privileged access. Much of that machinery, however, is model-independent. Whether the *evaluator*—the code that turns predictions into a score—is publicly documented and runnable is a property of the release, not of any model, and it can be checked without spending a single inference call.

We study that checkable property directly. Our central question is: *given only a benchmark’s paper and its public supplementary materials, repositories, datasets, documentation, and leaderboards, can an independent researcher reconstruct the benchmark’s evaluation protocol and run its evaluator without foundation-model inference?* We deliberately do not attempt to reproduce leaderboard scores, and we do not equate a benchmark’s public reproducibility with its scientific merit. A benchmark may be valuable and yet ship an evaluator that a third party cannot run; our audit measures the second property and reports it as such.

Data-science benchmarks make this question especially pointed. Their tasks span the knowledge-discovery workflow [5]—data access, cleaning, transformation, analysis, modeling, and reporting—so their evaluators are rarely a single accuracy function. They compare execution results against reference environments, parse notebook or SQL outputs, invoke database engines, grade against rubric files, or aggregate per-task scorers. Each of these steps is a place where a consequential detail (the exact metric, the tie-breaking rule, the mapping from raw predictions to an aggregate number, the expected input schema) can be omitted from public artifacts without anyone noticing, because the leaderboard still populates for the original authors who hold the missing piece.

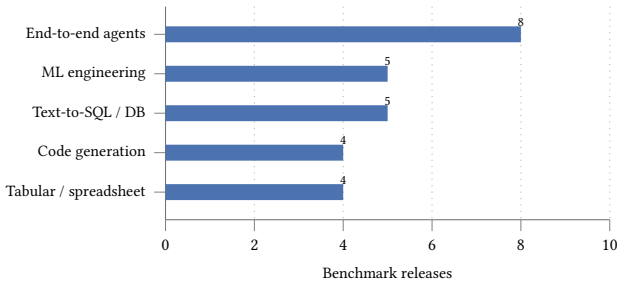


Figure 1: Composition of the audited corpus: 26 frozen benchmark releases across five data-science categories. Each release is pinned to a repository commit and audited on 13 July 2026.

We screen 35 candidate releases and audit a primary cohort of 26 whose official repositories were resolved and frozen at a repository commit on a single date (13 July 2026). The audit cohort spans five categories—data-science code generation, text-to-SQL and data-base agents, tabular and spreadsheet analysis, machine-learning experimentation and engineering, and end-to-end data-science agents (Figure 1)—selected through a documented, PRISMA-style screening procedure [14]. For each release we separate three outcomes that benchmark-quality work often merges into one: whether the artifact is *accessible*, whether the evaluation is *reconstructable* from public evidence, and whether the evaluator is *executable* under a controlled CPU-only protocol. We then assess every release and attempt every reachable evaluator under a pre-specified two-pass smoke test—first strictly as documented, then, in a separate pass, with minimal repairs that we release and explain—so that unassisted and repaired execution are never merged.

The screening and audit expose two distinct gaps. The completed search resolves official repositories for 30 of 35 candidates. Because four were recovered after the primary outcomes were known, we preserve the initial 26-release cohort and report those four separately as an expansion sensitivity. Within the primary cohort, reconstructability drops sharply—only 13 of 26 releases (50.0%) expose enough of their consequential evaluation details to be instantiated without unsupported guesses, and under a strict reading only 1 does. Executability drops further: only 7 of 26 evaluators (26.9%) produce a score after allowing documented repairs, and only 5 do so as documented. Because inter-model reliability misses its target and repair decisions involve auditor discretion, we treat R2 and after-repair E3 as descriptive sensitivity analyses; first-pass execution is the least discretionary endpoint. The gaps are specific and actionable. The most frequent reconstruction blocker is not absent code but an undocumented mapping from raw predictions to an aggregate score, followed by unstated grading and tie-breaking rules. The most frequent execution blocker is runtime dependence on credentials, hosted services, or required data that the public release does not include.

Contributions. This paper makes four contributions, all completed and released.

- (1) We document a 35-candidate screening frame, a public-artifact audit of 26 frozen releases, and a four-release

post-freeze expansion spanning five data-science categories, each frozen at a commit and a single audit date with a full evidence trail (Section 3).

- (2) We separate evaluation reconstruction into artifact access, specification reconstruction, and evaluator execution, using a frozen codebook and three-provider model panel rather than a single benchmark-quality score (Sections 3 and 5).
- (3) We perform CPU-only evaluator assessments for the entire cohort, reaching an executable command for 21 releases and documenting both unassisted and minimally repaired execution with reproducible logs and a failure taxonomy (Sections 4 and 5).
- (4) We release a public evidence resource—the corpus registry, line-level evidence records, raw and aggregated model judgments, execution logs, minimal-repair patches, and the analysis pipeline—so that authors, users, and reviewers can reuse and extend the audit (Section 8).

Throughout, we hold to a strict evidential discipline: we report what we could and could not find in public artifacts, we distinguish missing documentation from broken links and from inaccessible-but-documented artifacts, and we never infer that a procedure was not performed merely because we did not find public evidence of it.

2 Related Work

Documentation and evaluation reporting. Datasheets [6], Model Cards [13], BenchmarkCards [18], EvalCards [4], and Evaluation Cards [7] standardize what authors should disclose. ReproEvalCard [15] is closer: it audits 55 LLM-pipeline papers before proposing a reporting standard for prompts, judge configurations, retrieval snapshots, and traces. We instead treat the *benchmark release* as the unit, inspect repository evidence at a frozen commit, and attempt to execute the released scorer without model inference.

Benchmark assessment and meta-evaluation. A second line assesses benchmark quality. BetterBench [17] uses a lifecycle checklist; MEQA [19] provides a meta-evaluation framework for question-answering benchmarks. These works largely rely on documentation and design review. We differ in method and object: our primary evidence is repository-level inspection and controlled execution of the evaluator, not a quality rubric, and our outcomes are deliberately narrow (accessible / reconstructable / executable) rather than a single goodness score.

Execution-grounded reproducibility. The NeurIPS reproducibility program [16] and REPRO-Bench [8] emphasize executable artifacts. Rollout Cards [12] goes beyond reporting guidance: it audits 50 repositories, preserves rollout records, and shows that re-grading fixed outputs under different reporting rules can shift scores substantially. Our scope is narrower in one respect—we do not re-grade model outputs—and broader in another: we test whether heterogeneous data-science benchmark scorers can be invoked at all under a common zero-inference CPU protocol.

Task-level and automated auditing. Shortcut, contamination, and text-to-SQL audits inspect task content or labels [2, 3, 9, 10, 21]. Auto Benchmark Audit [20] is the closest automated counterpart: it

executes task-level audits across 168 benchmarks and validates detected specification, environment, and grading issues with experts. Our object and evidence differ: we audit release-level public evaluation interfaces, record claim–evidence provenance, and separate as-documented from repaired scorer execution. Task correctness and public scorer reconstructability remain complementary properties.

The gap we address. The closest work now overlaps substantially with individual pieces of our design: ReproEvalCard audits reporting, Rollout Cards audits repositories and re-grades outputs, and Auto Benchmark Audit executes task-level checks. We do not claim that repository auditing or execution is new in isolation. The incremental contribution is a release-level, evidence-linked measurement of whether public data-science benchmark scorers can be reconstructed and invoked, with frozen commits and an explicit separation between as-documented and auditor-repaired runs. This narrower positioning is the basis for our claims.

3 Corpus and Audit Protocol

3.1 Search and screening

We assembled candidate benchmarks for data-science tasks from recent foundation-model evaluation literature and community leaderboards, then applied a documented, PRISMA-style screening procedure [14]. Topical eligibility did not require a repository: candidates had to define a public benchmark for a foundation model or model-based agent over a data-science workflow before the search cutoff. The initial search resolved an official repository and immutable commit for 26 releases, which form the frozen primary audit cohort. A post-freeze search, performed after primary outcomes were known, resolved four more repositories. We fully code and smoke-test those four under the same protocol but report them as an expansion sensitivity rather than retrospectively changing the primary cohort. Five candidates remain unresolved. R2 and E3 are conditional estimates, not population prevalence estimates. The full search and amendment ledgers are released.

3.2 Corpus composition

The repository-resolved cohort contains **26 benchmark releases** across five categories (Figure 1): 4 data-science code generation, 5 text-to-SQL and database agents, 4 tabular and spreadsheet analysis, 5 machine-learning experimentation and engineering, and 8 end-to-end data-science agents. The unit of analysis is a *benchmark release at a frozen version*: each entry is pinned to a repository commit and audited on a single date (13 July 2026). Per-release commits, arXiv identifiers, and the audit date are given in the released registry.

3.3 Operational outcomes

We use four operational levels that separate concerns commonly merged in benchmark-quality work:

- A0 – Described:** a public paper or stable technical report defines the benchmark.
- A1 – Accessible:** within the repository-resolved cohort, core referenced artifacts are publicly reachable without privileged or paid access.

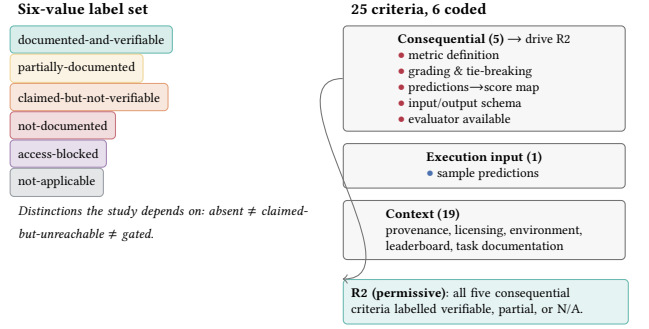


Figure 2: Structure of the frozen audit instrument (codebook v1.0.0): the six-value label set, the 25 criteria grouped by theme, and the five consequential evaluation criteria whose labels determine the R2 outcome.

R2 – Reconstructable: the consequential evaluation details can be instantiated without unsupported guesses.

E3 – Smoke-testable: the evaluator processes supplied or schema-valid synthetic predictions and produces a score or structured result in the controlled CPU environment (Section 4).

These levels measure *public evaluation reproducibility*. They are not a universal ranking of benchmark quality, and a benchmark can score well on its intended task while ranking low here.

3.4 Codebook and evidence protocol

The audit instrument is a frozen codebook (version 1.0.0, SHA-256 prefix aa57e2b3) with 25 criteria and a six-value label set: *documented-and-verifiable*, *partially-documented*, *claimed-but-not-verifiable*, *not-documented*, *not-applicable*, and *access-blocked* (Figure 2). The label set is deliberately built to preserve distinctions the study depends on: a missing artifact (*not-documented*) is separated from a claimed-but-unreachable one (*claimed-but-not-verifiable*) and from a gated one (*access-blocked*). For every non-trivial decision we record the benchmark and release identifier, the frozen commit, a source URL or repository path, a file or section pointer, the access date, a concise evidence note, each raw judgment, the deterministic aggregate, and the reason for any disagreement or repair. For *not-documented* decisions we additionally record the artifact locations and search terms inspected, so that “we did not find it” is itself auditable. We prefer canonical and primary sources in a fixed priority order (official paper and appendix, official supplementary material, official repository, official dataset or model-hub page, official documentation, official leaderboard documentation); third-party re-implementations may provide context but never establish that the *official* release is reconstructable.

R2 uses five consequential criteria: metric, grading rules, score aggregation, input/output schema, and evaluator availability. A release is R2-Reconstructable under our frozen rule when each of these five is *documented-and-verifiable*, *partially-documented* (a bounded gap), or *not-applicable*, and is *not* blocked by a *claimed-but-not-verifiable* or *not-documented* label. We report this permissive

rule as one descriptive R2 estimate and a conservative variant (requiring all five to be fully *documented-and-verifiable*) as a sensitivity check in Section 5. Availability of sample predictions/trajectories is treated as an input to the execution study, not as an R2 criterion.

3.5 Model-judge panel and aggregation

The six coded criteria yield 156 benchmark–criterion cells. Three model judges from independently operated providers—GPT-5.1, Claude Sonnet 5, and Gemini 3.5 Flash—each labeled every cell independently (468 raw decisions). Judges received the same pseudonymous, line-addressable frozen evidence packets and prompt, with no benchmark names, browsing, tools, other-judge outputs, or aggregate outcomes. Every decision required an evidence pointer validated against packet line bounds. Exact model snapshots, prompts, schemas, packets, raw responses, and usage ledgers are released.

We use no model adjudicator. An exact majority determines the aggregate label; a three-way tie is resolved deterministically toward the more conservative label on the majority pass/fail side. We report permissive, strict, unanimous, pairwise, leave-one-judge-out, and test–retest sensitivities. A Gemini default-thinking preflight was excluded wholesale after output truncation; all 26 Gemini production packets were then rerun with a frozen low-thinking setting. The release preserves the excluded pilot and a timeout-only repeat-run amendment.

Across all six criteria, nominal Krippendorff’s α was 0.471 (95% CI [0.373, 0.548], bootstrap clustered by benchmark), below the pre-specified 0.80 target. For the binary permissive boundary on the five R2 criteria, $\alpha = 0.622$, Gwet’s AC1 was 0.806, pairwise agreement was 86.2–88.5%, and 80.8% of cells were unanimous. On a preselected six-release repeat sample, exact and binary test–retest agreement were 84.3% and 93.5%, respectively. These are *inter-model* and test–retest statistics, not human inter-rater reliability or evidence of ground truth. Because the nominal reliability gate was missed, R2 remains LLM-assisted descriptive coding; A1 and E3 carry the paper’s primary evidential weight.

4 CPU-Only Execution Study

Reconstructability (R2) asks whether the evaluation *could* be instantiated from public evidence; executability (E3) asks whether the official evaluator *actually runs*. We assess all 26 releases under a pre-specified, zero-inference protocol; for 21, a public evaluator interface was reached and a command was run. The remaining five were classified before execution because required data, credentials, or an applicable standalone scorer were unavailable.

4.1 What E3 tests, and what it does not

The authoritative question for E3 is narrow: can the benchmark’s *scorer*—not the model or agent under evaluation—process supplied or schema-valid synthetic predictions and emit a score or structured result on CPU. We never run a foundation model, never reproduce a leaderboard number, and never use benchmark answers to construct predictions. A passing smoke test demonstrates that the evaluation apparatus is publicly runnable; it does *not* demonstrate full computational reproducibility of any reported result, and we are explicit about that distinction throughout.

4.2 Controlled environment and resource limits

All runs used a single clean, documented CPU environment: a conda environment (e3smoke, Python 3.11.15) whose exact package versions are released as a pip freeze. There were 0 GPU-hours, 0 paid model-API calls, and no private credentials. We fixed a per-attempt wall-clock resource cap and recorded, for every attempt, the exact commands, resolved dependencies, runtime, exit status, outputs, and any failure. One environment-level setting was neutralized uniformly *before* the first pass rather than attributed to individual benchmarks: the sandbox’s PYTHONSAFEPATH default suppresses the current-directory import that several repositories assume, so we cleared it as part of defining the controlled environment.

4.3 Inputs

For the first attempt we used official sample predictions when a release shipped them. Otherwise we supplied *schema-valid synthetic predictions* constructed only from the evaluator’s documented input schema—never from benchmark answers—so that a pass reflects the scorer’s ability to process well-formed input, not our knowledge of the correct outputs. Synthetic-input construction and the resulting files are released with the execution logs.

4.4 Two-pass protocol

Each evaluator was run under a strict two-pass protocol, and the two passes are never merged.

- (1) **First pass (as documented).** We followed the official instructions on the frozen commit and did not modify benchmark code. The outcome of this pass alone determines the *unassisted* (first-pass) E3 count.
- (2) **Minimal-repair pass (separate).** Only if the first pass failed did we attempt a separate minimal-repair pass. Repairs were confined to the smallest change that let the scorer run without altering its scoring logic—for example, an in-process execution shim with a signal-based timeout, or granting a specific documented network fetch. Every repair patch is released and explained, and repaired passes are reported separately from unassisted ones.

4.5 Outcome taxonomy

Each first-pass attempt receives one label from a fixed taxonomy separating passes, missing evaluators or data, access or service blocks, dependency or resource failures, unsupported schemas, and non-applicable cases. Exact labels and rules are released. A release counts toward E3 (any) if it passes as documented or after minimal repair; the distinction between the two is preserved in every downstream count.

The frozen research contract originally designated E3 after either pass as the primary endpoint. Because repair eligibility and semantic equivalence require auditor judgment, this revision promotes the as-documented first pass as the least discretionary execution estimate and treats E3 after repair as a secondary sensitivity analysis. This post-freeze hierarchy change is disclosed in the released protocol amendment; both counts remain visible.

Table 1: Screening and conditional audit outcomes (Wilson 95% intervals).

Outcome	k	n	Proportion	95% CI
Official repository resolved	30	35	85.7%	[70.6%, 93.7%]
R2 Reconstructable (permissive)	13	26	50.0%	[32.1%, 67.9%]
R2 Reconstructable (strict)	1	26	3.8%	[0.7%, 18.9%]
E3 as documented	5	26	19.2%	[8.5%, 37.9%]
E3 after documented repair	7	26	26.9%	[13.7%, 46.1%]

4.6 Ethical and licensing safeguards

We used only public artifacts under their stated access conditions, and we did not redistribute third-party data or code where licenses prohibit it; the released resource contains our evidence records, logs, and patches, and points to official artifacts rather than mirroring them. We treated credential-gated or service-gated evaluators as *blocked* outcomes rather than attempting to circumvent access controls, and we followed the pre-specified resource caps and stopping rules uniformly across the corpus.

5 Results

We report screening yield, reconstructability, and evaluator executability, then claim–evidence concordance and failure patterns. Repository resolution uses the 35-candidate screening frame; primary downstream outcomes use the 26-release frozen cohort, with a four-release expansion reported separately. Proportions carry 95% Wilson intervals. Unassisted and repaired execution are reported separately, and access- or service-blocked cases are kept distinct from software failures throughout.

5.1 Screening and artifact availability

The completed search resolved and froze an official repository for **30/35 candidates** (85.7%, 95% CI [70.6%, 93.7%]). Five unresolved candidates remain screening outcomes; we do not infer that no artifact exists. The primary artifact analysis remains tied to the 26 repositories frozen before outcomes were known. Within that cohort, all repositories and their referenced core artifacts were reachable at the audit date, but artifact presence was uneven (Figure 3). Evaluator code is present for all 26 releases (100%) and a dataset for 25 (96.2%), but sample predictions are present for only 13 (50.0%), a container recipe for only 7 (26.9%), an environment specification for 21 (80.8%), and an explicit license for 23 (88.5%). The sparse classes—sample predictions and containers—are precisely the artifacts that most directly govern whether an evaluator can be reconstructed and run. These artifact-class rates describe the primary frozen cohort.

Post-freeze expansion sensitivity. All four post-freeze releases reach A1. Three reach R2 and produce a first-pass CPU score on schema-valid synthetic predictions; the fourth releases data and schemas but no evaluator. Combining this declared extension with the primary cohort gives descriptive R2 of 16/30, first-pass E3 of 8/30, and repaired E3 of 10/30. The direction of the primary finding is unchanged. Frozen commits, criterion evidence, smoke commands, and logs are released; no primary label is changed.

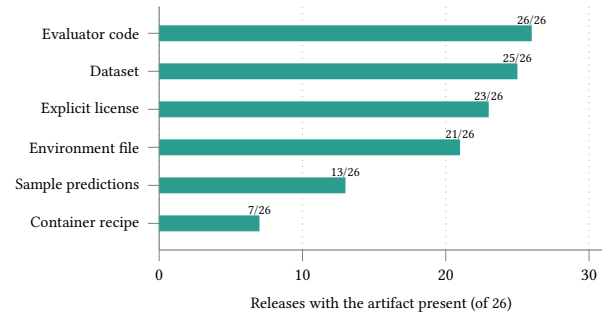


Figure 3: Presence of specific artifact classes across the 26 releases. Evaluator code and datasets are near-universal; sample predictions and container recipes—the artifacts most tied to reconstruction and execution—are the sparsest.

5.2 Reconstructability (R2)

Under the frozen permissive rule, **13 of 26 releases** (50.0%, 95% CI [32.1%, 67.9%]) are R2-Reconstructable: their five consequential evaluation criteria are each documented-and-verifiable, partially-documented, or not-applicable. Under the strict rule that requires all five to be fully documented-and-verifiable or not-applicable, only **1 of 26** (3.8%, 95% CI [0.7%, 18.9%]) qualifies. Requiring permissive agreement from all three judges yields 7/26. The gap between these readings is itself a finding: R2 is highly sensitive to how partial documentation is treated. Because inter-model reliability missed its pre-specified gate, both estimates are descriptive rather than validated prevalence estimates. Across leave-one-judge-out analyses, pessimistic two-judge R2 ranges from 7 to 10 and optimistic R2 from 14 to 17. Figure 4 shows the per-criterion label matrix that drives these outcomes.

5.3 Evaluator executability (E3)

The least discretionary execution estimate is the as-documented first pass: **5/26 (19.2%, 95% CI [8.5%, 37.9%])** produced a score without repair. Two additional evaluators ran only after documented interventions, giving a secondary after-repair estimate of **7/26 (26.9%, 95% CI [13.7%, 46.1%])**. All 26 releases were assessed, but an evaluator command was actually run for 21; the remaining five were classified before execution because required data, credentials, or an applicable standalone scorer were unavailable. Conditioning on the 21 releases where a public evaluator interface was reached raises first-pass E3 to 5/21 (23.8%, 95% CI [10.6%, 45.1%]) and after-repair E3 to 7/21 (33.3%, 95% CI [16.3%, 56.3%]). We report the 26-release denominator as primary because reaching an evaluator interface at all is part of what the audit measures, but both denominators are released. The seven eventual passes were QRData, InfiAgentDABench, ScienceAgentBench, DSbench, and SpreadsheetBench (unassisted) and DS-1000 and TableBench (after minimal repair). Executability by category is uneven (Figure 5): tabular 2/4, end-to-end agents 3/8, code generation 1/4, machine-learning engineering 1/5, and text-to-SQL 0/5. No text-to-SQL evaluator ran to a score under the public-only protocol, most often because the scorer required database files or gold artifacts not shipped with the frozen release.

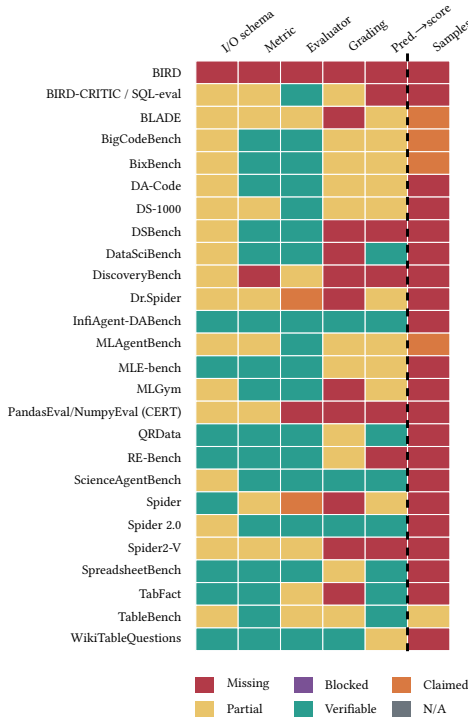


Figure 4: Deterministic three-judge aggregate labels for the five R2 criteria (left of the dashed line) plus the sample-predictions input (right) across all 26 releases. A release is R2-Reconstructable when none of its five criteria is *claimed-but-not-verifiable* or *not-documented*.

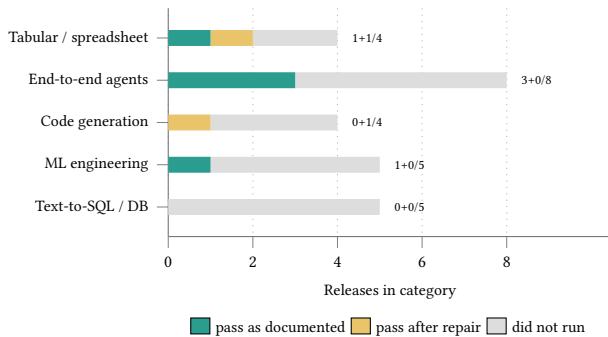


Figure 5: Evaluator executability by category. Bars show the number of releases reaching E3 out of the category total; unassisted and after-repair passes are distinguished.

5.4 Claim–evidence concordance

For the 14 releases whose README frames a directly runnable evaluation, we asked whether that framing held under our public-only, CPU-only protocol. Only 2 of 14 (14.3%, 95% CI [4.0%, 39.9%]) actually ran to a score; the other 12 were framed as runnable but were gated at runtime—6 by dependency or environment failures, 4

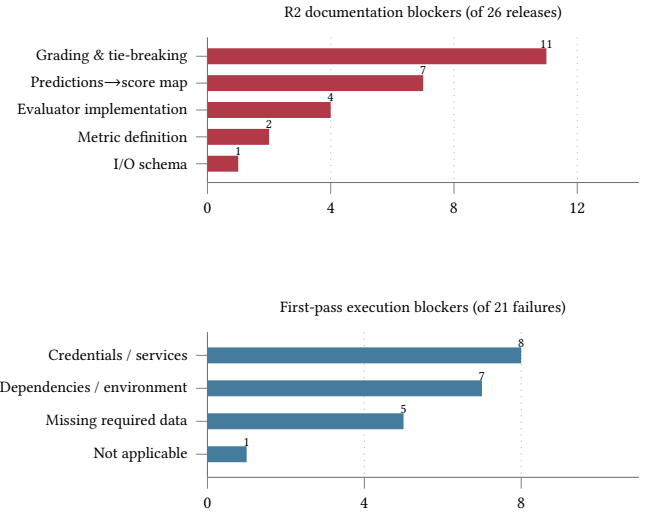


Figure 6: Reconstruction blockers (R2 criteria failing) and first-pass execution blockers (failure taxonomy) across the corpus. The leading reconstruction barrier is undocumented grading and tie-breaking; the leading execution barrier is runtime dependence on credentials or services.

by credential or service requirements, and 2 by missing data or non-applicability. We frame this strictly as a concordance gap between the public framing and the observed behavior under our protocol, not as a claim that the authors’ own environments do not run.

5.5 Failure patterns and category differences

The reconstruction and execution gaps have specific, recurring causes (Figure 6). Among R2 blockers, the most frequent is undocumented *grading and tie-breaking rules* (11 releases), followed by an absent *predictions-to-aggregate-score mapping* (7), an absent *evaluator implementation* in the inspected artifacts (4), an underspecified *metric definition* (2), and an unclear *input/output schema* (1)—note that a single release can be blocked on more than one criterion. Missing code is thus not the primary reconstruction barrier; missing *specification of how existing code turns predictions into a number* is. Across the 21 first-pass failures, the taxonomy is led by credential- or service dependence (8), dependency- or environment failures (7), and missing required data (5), with one not-applicable case. Two of these—TableBench (credentials) and DS-1000 (environment)—later passed under minimal repair, so the 19 releases that never produced a score distribute 7, 6, 5, and 1 over the same categories.

We also examined the association between reconstructability and executability. R2 and E3 are strongly associated (odds ratio ≈ 10.3 , Fisher exact $p = 0.073$): reconstructable releases are far more likely to be executable, though at $N = 26$ this does not reach significance at $\alpha = 0.05$ and we treat it as exploratory. As a robustness check, restricting to the 18 releases whose first-pass failure was *not* a service- or access-block raises the executable proportion to 6/18 (33.3%, 95% CI [16.3%, 56.3%]), indicating that a substantial

Table 2: Descriptive reconstructability (R2) and smoke-testability (E3) by benchmark category.

Category	n	R2	E3	E3 proportion
code-gen	4	3	1	25%
e2e-agent	8	4	3	38%
ml-eng	5	2	1	20%
tabular	4	3	2	50%
text2sql	5	1	0	0%

share of the execution gap is attributable to runtime access dependencies rather than to broken scoring code. A category comparison of text-to-SQL against the rest is not significant (Fisher $p = 0.28$) and is reported only as exploratory. Per-release outcomes are released in `table3_per_benchmark.csv`; per-category endpoints are in Table 2.

Artifact presence does not predict either endpoint. Because our recommendations to authors concern which artifacts to ship, we tested directly whether shipping them is associated with better outcomes. It is not. Releases that publish sample predictions are no more reconstructable than those that do not (6/13 vs. 7/13, Fisher $p = 1.0$) and no more executable on the first pass (2/13 vs. 3/13, $p = 1.0$). Shipping a container recipe is, if anything, inversely related to execution: 0/7 containerized releases produced a first-pass score against 5/19 without ($p = 0.28$). An environment file shows no association either (3/21 vs. 2/5, $p = 0.24$). None of the nine tests approaches significance at $N = 26$, and we draw no causal conclusion; the defensible reading is that *presence of an artifact class is not sufficiency of the evidence it is meant to supply*. A container that provisions an agent runtime does not make a scorer invocable without credentials, and a sample-predictions file whose expected score is undocumented still leaves the predictions-to-score mapping unstated. This tempers our recommendations (Section 6), which rest on the observed blocker mechanisms rather than on any measured association.

Shared lineage does not drive the headline. Our intervals treat releases as exchangeable, but several share authorship or an evaluator codebase (Section 7). Collapsing the Spider, BIRD, and DS-1000/CERT families leaves 21 independent lineages. First-pass E3 is stable under every adjustment: 5/22 (22.7%) dropping the Spider family, 5/24 (20.8%) dropping BIRD, 5/24 dropping DS-1000/CERT, and 5/21 (23.8%) retaining one release per lineage. R2 likewise moves only from 13/26 (50.0%) to 12/21 (57.1%). Lineage clustering therefore inflates our effective sample size but does not manufacture the gap we report.

5.6 Inter-model agreement as a finding

The three judges exactly agreed on 46.8% of all 156 cells; nominal Krippendorff’s $\alpha = 0.471$ (95% CI [0.373, 0.548]), below the pre-specified 0.80 target. Agreement is materially stronger at the binary boundary used by permissive R2: among the 130 R2 cells, binary $\alpha = 0.622$, AC1=0.806, pairwise agreement is 86.2–88.5%, and 80.8% are unanimous. The preselected repeat sample gives

84.3% exact and 93.5% binary test–retest agreement. These sensitivities do not replace the failed nominal gate. They show that much of the disagreement concerns documentation degree while some benchmark-level R2 classifications remain judge-dependent. We therefore report majority 13/26, strict 1/26, unanimous 7/26, and leave-one-out bounds together, retain R2 as descriptive, and release every raw output and aggregation rule.

6 Case Studies and Recommendations

The aggregate endpoints in Section 5 summarize *what* fraction of the corpus reaches each level; the three cases below illustrate *why* releases land where they do, drawn to span the ladder from a clean pass to a reconstructable-but-blocked release. Each case reports only what we observed in public artifacts at the frozen commit; we do not infer author intent, and we distinguish an evaluator that ran under our protocol from one we merely inspected.

6.1 QRData: reconstructable and executable as documented

QRData (end-to-end agent category, commit de450af) reached all four levels without repair. Its evaluator at `benchmark/eval.py` reads a predictions file, compares numeric answers against reference values under a documented tolerance (`error_scale = 0.03`), and prints an accuracy—a predictions-to-score mapping that is fully specified in public artifacts. We supplied four schema-valid synthetic predictions that do not depend on the benchmark’s answers; the scorer ran on CPU in 0.04 s (*pass-as-documented*). QRData is the shape of release the audit rewards: the five consequential evaluation details (metric, grading, predictions-to-score mapping, input/output schema, and an available evaluator implementation) are all present and mutually consistent, so an independent researcher can instantiate and exercise the scorer with no unsupported guesses.

6.2 DS-1000: reconstructable, executable only after a documented repair

DS-1000 (data-science code generation, frozen commit b39aab7) is reconstructable (all five R2 criteria satisfied) but did *not* run on the first pass: its execution harness calls `ProcessPoolExecutor`, which raised a `PermissionError` during a POSIX semaphore query inside our controlled sandbox—a property of the execution environment, not of the benchmark’s scoring logic. We recorded this as *dependency-or-environment-failure* on the unassisted pass, then applied a minimal repair in a separate pass: an in-process execution shim using a signal-based timer instead of subprocess isolation, run over a 50-problem smoke slice with no edit to the repository. The repaired pass produced a real per-split score (*pass-after-minimal-repair*). DS-1000 illustrates why we never merge first-pass and repaired outcomes: counting it as an unqualified success would hide a reproducibility-relevant environment dependency, while counting it as a failure would misattribute a sandbox artifact to the benchmark. The released patch and both logs let a reader see exactly what changed. Because serial execution removes process isolation, we do not claim semantic equivalence to the official harness and count this case only in the repaired sensitivity result.

6.3 MLE-bench: reconstructable but not executable under the protocol

MLE-bench (ML engineering, frozen commit 507f92e) satisfies all five R2 criteria—its per-competition graders are concrete, and the metric and predictions-to-score mapping are specified—yet it did not reach E3. Importing the grading module pulls in the package’s utility layer, which requires `py7zr`; under our fixed dependency set this raised a `ModuleNotFoundError`, which we classified as *dependency-or-environment-failure*. Following our reporting convention, we state that *the evaluator did not run under our controlled protocol*, not that it is broken: a differently provisioned environment may well execute it. MLE-bench is the paper’s central pattern in miniature—reconstructability on paper does not guarantee zero-inference executability—and it is one of the cases behind the strong but non-significant R2–E3 association (Section 5).

6.4 Recommendations

The failure structure in Figure 6 and the released per-release table points to low-cost changes that address the blockers we observed, without any change to a benchmark’s difficulty or scientific goals. We state their status plainly: these are mechanistic remedies aimed at specific recorded failures, not interventions validated by our data. Merely *shipping* any of these artifact classes shows no association with reaching R2 or E3 in our corpus (Section 5), so each recommendation below is about the *content* the artifact must carry, not its presence in the repository.

- **Ship sample predictions with their expected score.** Half the corpus (13/26) publishes no example predictions, yet the most common reconstruction gaps are undocumented grading rules (11/26) and predictions-to-aggregate-score mapping (7/26). Note that publishing sample predictions alone did not predict reconstructability (6/13 vs. 7/13): a prediction file only closes these gaps when it is paired with the score it should produce, which pins down the mapping and the grading rule at once.
- **Separate the scorer from the runtime.** Most E3 blockers are credential, service, or environment dependencies encountered while importing or invoking the evaluator, not defects in the scoring logic. A scorer that accepts a predictions file and emits a score without network access, credentials, or heavyweight optional dependencies is independently checkable.
- **Pin an environment for the scorer, not just the agent.** Only 27% (7/26) ship a container and 81% (21/26) any environment file—but no containerized release produced a first-pass score (0/7). The containers we found provision heavy agent runtimes and frequently presuppose credentials or services, so they do not make the scorer independently invocable. A pinned, evaluation-only environment specification, distinct from the agent-execution environment, is what would remove the dependency failures we observed.
- **Document grading edge cases.** Grading rules and tie-breaking were R2-blocking for 11/26 releases. Stating how partial, malformed, or tied predictions are scored converts a guess into a specification.

These recommendations follow from the observed blockers; they are offered to benchmark authors, users, and reviewers as concrete, artifact-level steps, not as a judgment of any benchmark’s overall quality and not as claims validated by our corpus. Testing whether they in fact raise reconstructability or executability requires a prospective study of releases adopting them, which our released instrument is designed to support.

7 Limitations

Model judges, and no human gold standard. Our three judges are language models operated by different providers, not human annotators. Shared training data or conventions can create correlated blind spots, so inter-model agreement is neither correctness nor human inter-rater reliability. R2 is explicitly LLM-assisted descriptive coding, not ground truth. We stress the sharpest form of this limitation: no subset of the 156 cells was independently coded by a human, so the panel’s *accuracy* is unmeasured and only its *consistency* is reported. At this corpus size a human-coded gold standard is tractable, and its absence—not any resource constraint—is why R2 cannot carry primary evidential weight here. The released packets, prompt, and per-cell aggregates are structured so that such a validation can be added directly against our frozen labels; A1 and E3, which do not depend on the panel, carry the paper’s primary weight in the meantime.

Reliability below the pre-specified target. Krippendorff’s α (nominal) was 0.471, below our pre-specified ≥ 0.80 target. Binary agreement is stronger but also misses that alpha gate. We retain the target, release raw labels and deterministic aggregation, and report strict, unanimous, leave-one-out, and repeatability sensitivities.

Small corpus. The primary corpus has 26 releases. Category comparisons and the R2–E3 association are exploratory; intervals are wide and we make no significance or aggregate benchmark-quality claim.

Shared benchmark lineage. Several releases share authorship or codebase lineage—the Spider family (Spider, Spider2.0, Spider2-V, Dr.Spider), the BIRD family (BIRD, BIRD-CRITIC), and DS-1000 relative to CERT—leaving 21 independent lineages. Outcomes may therefore cluster, whereas our intervals treat releases as exchangeable, so the effective sample is smaller than $N = 26$ and the reported intervals are correspondingly optimistic. Point estimates are nonetheless stable under leave-one-family-out and one-per-lineage recomputation (Section 5). We retain separately published releases as units but interpret family-adjacent comparisons conservatively.

Repository-resolved cohort. The primary cohort is conditional on resolving an official repository and does not estimate prevalence among all eligible benchmarks. A completed post-freeze search raises resolution to 30/35; because outcomes were already known, the four additions form a declared sensitivity rather than alter the frozen cohort.

Smoke tests, not full reproduction. E3 only asks whether an evaluator returns a result on supplied or synthetic predictions on CPU. It neither reproduces leaderboard numbers nor runs model inference; scorer invocation is weaker than full computational reproducibility.

Single environment and repair discretion. E3 is measured in one controlled environment, so it is a property of the release *and* of that environment. This matters most for the 7 of 21 first-pass failures labelled dependency-or-environment: DS-1000 failed on a sandbox ProcessPoolExecutor restriction and MLE-bench on a missing optional dependency, and a differently provisioned host might well run both. Such cases are a ceiling on how much of the execution gap can be attributed to releases rather than to our provisioning; the credential-, service-, and missing-data blockers (13 of 21) do not share this weakness, and the service-block-excluded sensitivity (6/18) in Section 5 bounds the effect. *Minimal* repair is likewise judgmental. We applied environment handling uniformly, limited repairs to configuration and non-semantic shims, released every patch, and never merged first-pass with repaired outcomes. A multi-environment replication is the natural next step and the released smoke harness is written to support it.

Point-in-time snapshot. Outcomes are anchored to frozen commits and the 2026-07-13 audit date; they are not standing claims about evolving repositories.

Scope and historical releases. Pre-LLM benchmarks are included when later foundation-model evaluations use their scorers. A *not-documented* code records searched locations and terms; missing evidence is not evidence that a procedure was never performed.

8 Conclusion

We asked whether an independent researcher can reconstruct and invoke a data-science benchmark evaluator from public artifacts without model inference. We resolved official repositories for 30 of 35 screened candidates. The primary audit remains the 26 releases frozen before outcomes were known; four later resolutions form a declared expansion sensitivity. Three-provider model coding classifies 13/26 as reconstructable under a permissive majority rule, 1/26 under a strict rule, and 7/26 under unanimous permissive agreement, but below-target reliability makes these descriptive estimates. Execution is less ambiguous: 5/26 (19.2%) run as documented, while two additional cases run only after auditor interventions. The expansion produces consistent descriptive estimates: R2 16/30, first-pass E3 8/30, and repaired E3 10/30.

Within the cohort, missing predictions-to-score mappings and grading rules dominate R2 gaps; credentials, services, and environment dependencies dominate execution failures. The R2–E3 association is exploratory. These outcomes measure public reconstructability and scorer invocation, not benchmark quality or full reproduction.

We release the screening frame, frozen cohort, evidence and model-judgment records, execution logs, repair patches, and analysis pipeline at <https://github.com/nausicaa2701/reconstructing-benchmark-evaluations>. Benchmark authors can close common gaps by shipping sample predictions, isolating the scorer, pinning its environment, and documenting grading edge cases.

References

- [1] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. 2024. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. arXiv preprint arXiv:2410.07095.
- [2] Simin Chen, Yiming Chen, Zexin Li, Yifan Jiang, Zhongwei Wan, Yixin He, Dezhi Ran, Tianle Gu, Haizhou Li, Tao Xie, and Baishakhi Ray. 2025. Recent Advances in Large Language Model Benchmarks against Data Contamination: From Static to Dynamic Evaluation. arXiv preprint arXiv:2502.17521.
- [3] Chunyuan Deng, Yilun Zhao, Yuzhao Heng, Yitong Li, Jiannan Cao, Xiangru Tang, and Arman Cohan. 2024. Unveiling the Spectrum of Data Contamination in Language Models: A Survey from Detection to Remediation. arXiv preprint arXiv:2406.14644.
- [4] Ruchira Dhar, Danae Sanchez Villegas, Antonia Karamolegkou, Alice Schiavone, Yifei Yuan, Xinyi Chen, Jiaang Li, Stella Frank, Laura De Grazia, Monorama Swain, Stephanie Brandl, Daniel Hershcovich, Anders Søgaard, and Desmond Elliott. 2025. EvalCards: A Framework for Standardized Evaluation Reporting. arXiv preprint arXiv:2511.21695.
- [5] Usama Fayyad, Gregory Piatetsky-Shapiro, and Padhraic Smyth. 1996. From Data Mining to Knowledge Discovery in Databases. *AI Magazine* 17, 3 (1996), 37–54.
- [6] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. 2021. Datasheets for Datasets. *Commun. ACM* 64, 12 (2021), 86–92. doi:10.1145/3458723
- [7] Avijit Ghosh, Anka Reuel, Jenny Chim, Wm. Matthew Kennedy, Srishti Yadav, Jennifer Mickel, Yanan Long, Andrew Tran, A. Kornilova, Damian Stachura, Kevin Klyman, F. Friedrich, Jeba Sania, Max Lamparth, Jan Batzner, Anoop Mishra, Eliya Habbu, Yixiong Hao, N. Heath, Shalaleh Rismani, Usman Gohar, Andreas Loehr, David Manheim, Ruchira Dhar, Sree Harsha Nelaturu, Aarush Sinha, Leshem Choshen, Drishti Sharma, I. Khire, Amit Saha, Subramanyam Sahoo, Michael Hardy, M. A. Riegler, Kabir Manghnani, M. Lin, Yanan Jiang, Yilin Huang, Asaf Yehudai, Jessica Ji, A. Hofmann, Mubashara Akhtar, Nuno Moniz, Yacine Jernite, Stella Biderman, Zeerak Ta-lat, Sanmi Koyejo, M. Kochenderfer, and Irene Solaiman. 2026. Evaluation Cards: An Interpretive Layer for AI Evaluation Reporting. arXiv preprint arXiv:2606.09809.
- [8] Chuxuan Hu, Liyun Zhang, Yeji Lim, A. Wadhvani, Austin Peters, and Daniel Kang. 2025. REPRO-Bench: Can Agentic AI Systems Assess the Reproducibility of Social Science Research? arXiv preprint arXiv:2507.18901.
- [9] Tianyi Jin et al. 2026. Text-to-SQL Benchmarks are Broken: An In-Depth Analysis of BIRD and Spider 2.0-Snow. *CIDR*.
- [10] Divyansh Kaushik and Zachary C. Lipton. 2018. How Much Reading Does Reading Comprehension Require? A Critical Investigation of Popular Benchmarks. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 5010–5015. doi:10.18653/v1/D18-1546
- [11] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, Honolulu, Hawaii, USA, 18319–18345. <https://proceedings.mlr.press/v202/lai23b.html>
- [12] C. Masters, Ziyu Liu, and Stefano V. Albrecht. 2026. Rollout Cards: A Reproducibility Standard for Agent Research. arXiv preprint arXiv:2605.12131.
- [13] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. 2019. Model Cards for Model Reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*. Association for Computing Machinery, New York, NY, USA, 220–229. doi:10.1145/3287560.3287596
- [14] Matthew J. Page, Joanne E. McKenzie, Patrick M. Bossuyt, Isabelle Boutron, Tammy C. Hoffmann, Cynthia D. Mulrow, Larissa Shamseer, Jennifer M. Tetzlaff, Elie A. Akl, Sue E. Brennan, et al. 2021. The PRISMA 2020 Statement: An Updated Guideline for Reporting Systematic Reviews. *BMJ* 372 (2021), n71. doi:10.1136/bmj.n71
- [15] Priyaranjan Pattnayak and Apoorv Bhatia. 2026. ReproEvalCard: A Reporting Standard for Reproducible Evaluation of LLM Pipelines. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, San Diego, California, United States, 238–249. doi:10.18653/v1/2026.acl-short.22
- [16] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d’Alché Buc, Emily Fox, and Hugo Larochelle. 2021. Improving Reproducibility in Machine Learning Research: A Report from the NeurIPS 2019 Reproducibility Program. *Journal of Machine Learning Research* 22, 164 (2021), 1–49.
- [17] Anka Reuel, Amelia Hardy, Chandler Smith, Max Lamparth, Malcolm Hardy, and Mykel J. Kochenderfer. 2024. BetterBench: Assessing AI Benchmarks, Uncovering Issues, and Establishing Best Practices. arXiv preprint arXiv:2411.12990.
- [18] Anna Sokol, Elizabeth Daly, Michael Hind, David Piorkowski, Xiangliang Zhang, Nuno Moniz, and Nitesh Chawla. 2024. BenchmarkCards: Standardized Documentation for Large Language Model Benchmarks. arXiv preprint arXiv:2410.12974.
- [19] Jaime Raldua Veuthey, Zainab Ali Majid, S. Hariharan, and Jacob Haimes. 2025. MEQA: A Meta-Evaluation Framework for Question & Answer LLM Benchmarks. arXiv preprint arXiv:2504.14039.

- [20] Junlin Wang, Federico Bianchi, Shang Zhu, Fan Nie, Yongchan Kwon, Bhuwan Dhingra, and James Zou. 2026. Automated Benchmark Auditing for AI Agents and Large Language Models. arXiv preprint arXiv:2605.26079.
- [21] Cheng Xu, Shuhao Guan, Derek Greene, and M.-Tahar Kechadi. 2024. Benchmark Data Contamination of Large Language Models: A Survey. arXiv preprint arXiv:2406.04244.

A Audit Sheet Schema

Each audit decision in the released dataset is a record with the following fields, which together satisfy our evidence requirements (every non-trivial decision carries a benchmark and release identifier, a frozen commit, a source pointer, an access date, an evidence note, each raw judgment, and the deterministic aggregate where applicable):

- benchmark name;
- release identifier and frozen commit;
- arXiv identifier;
- task category;
- source URL or repository path;
- file, section, or line pointer;
- audit criterion;
- raw judgment (per model provider);

- aggregate label and deterministic rule;
- evidence quotation or factual note;
- disagreement/repair explanation;
- access date.

For not-documented decisions, the record additionally lists the artifact locations and search terms inspected. The full schema is released as `master_outcomes_schema.json`.

B Search Keywords

Corpus construction used the following search keywords across arXiv and code hosts; the full screening counts are reported with the corpus (Section 3):

- LLM data science benchmark
- foundation model data analysis benchmark
- LLM agent data science benchmark
- text-to-SQL benchmark LLM
- machine learning agent benchmark
- data science code generation benchmark
- benchmark contamination LLM
- benchmark reproducibility LLM